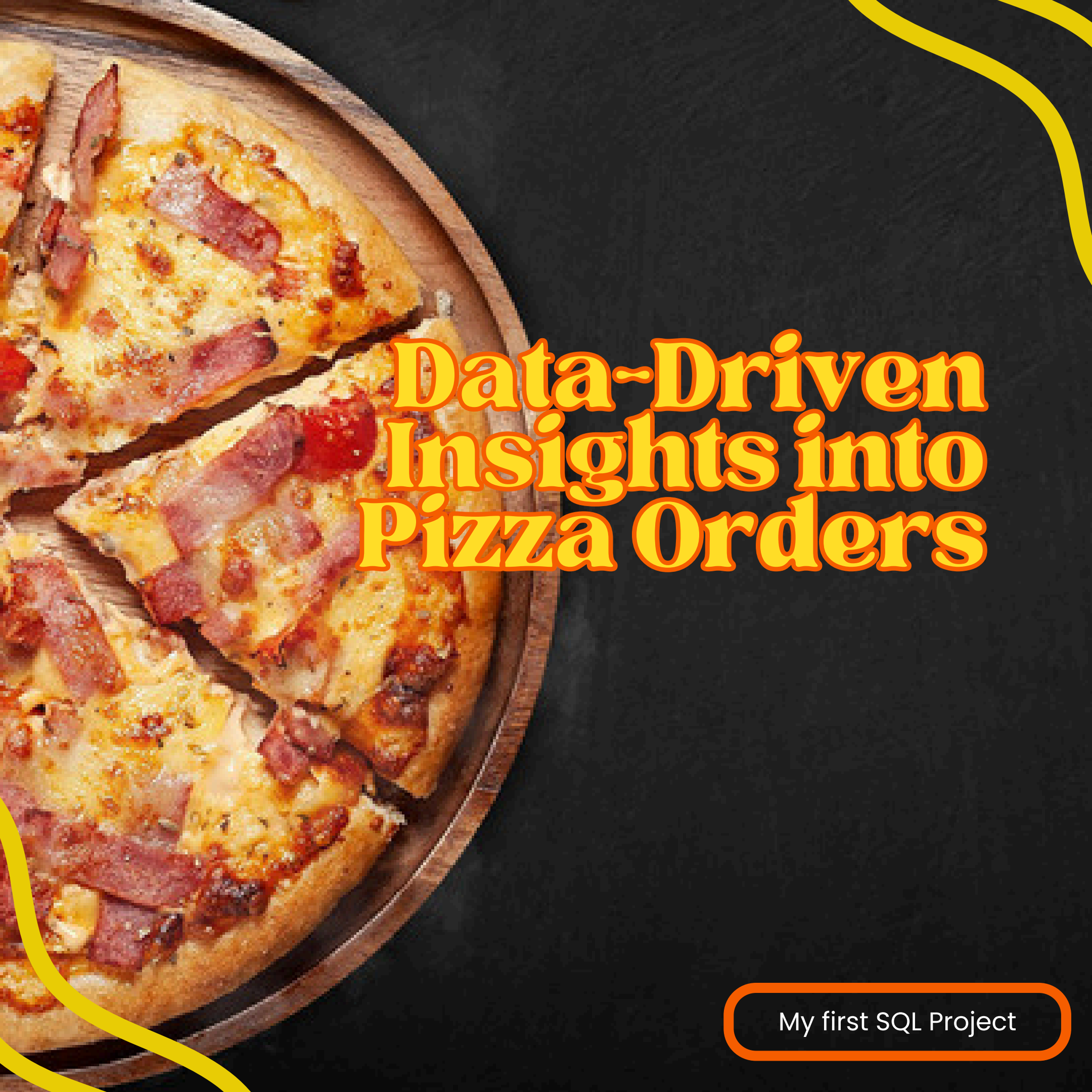


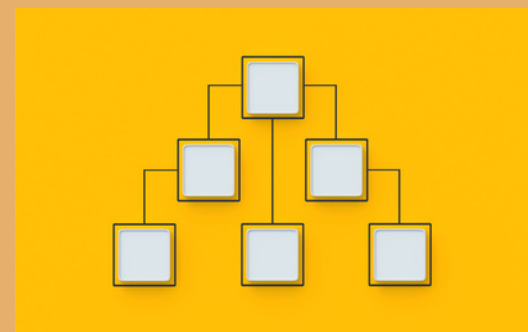
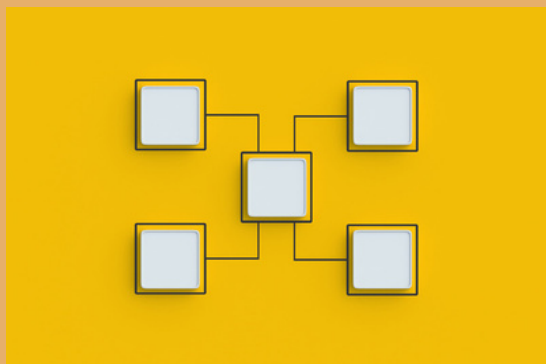
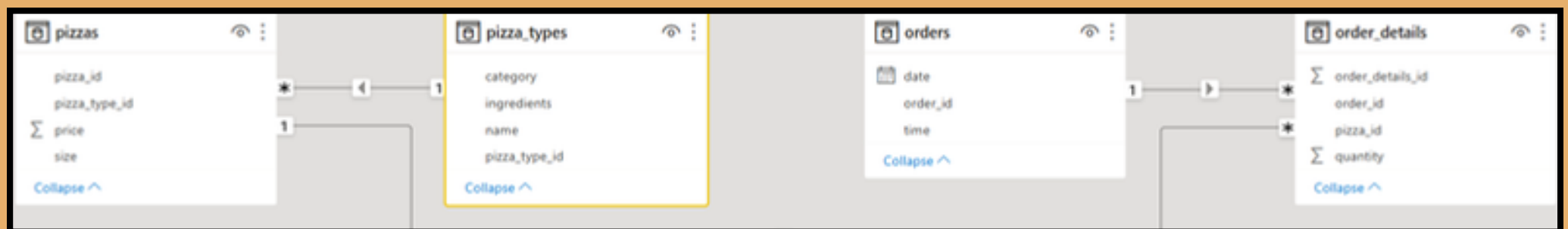


Data-Driven Insights into Pizza Orders

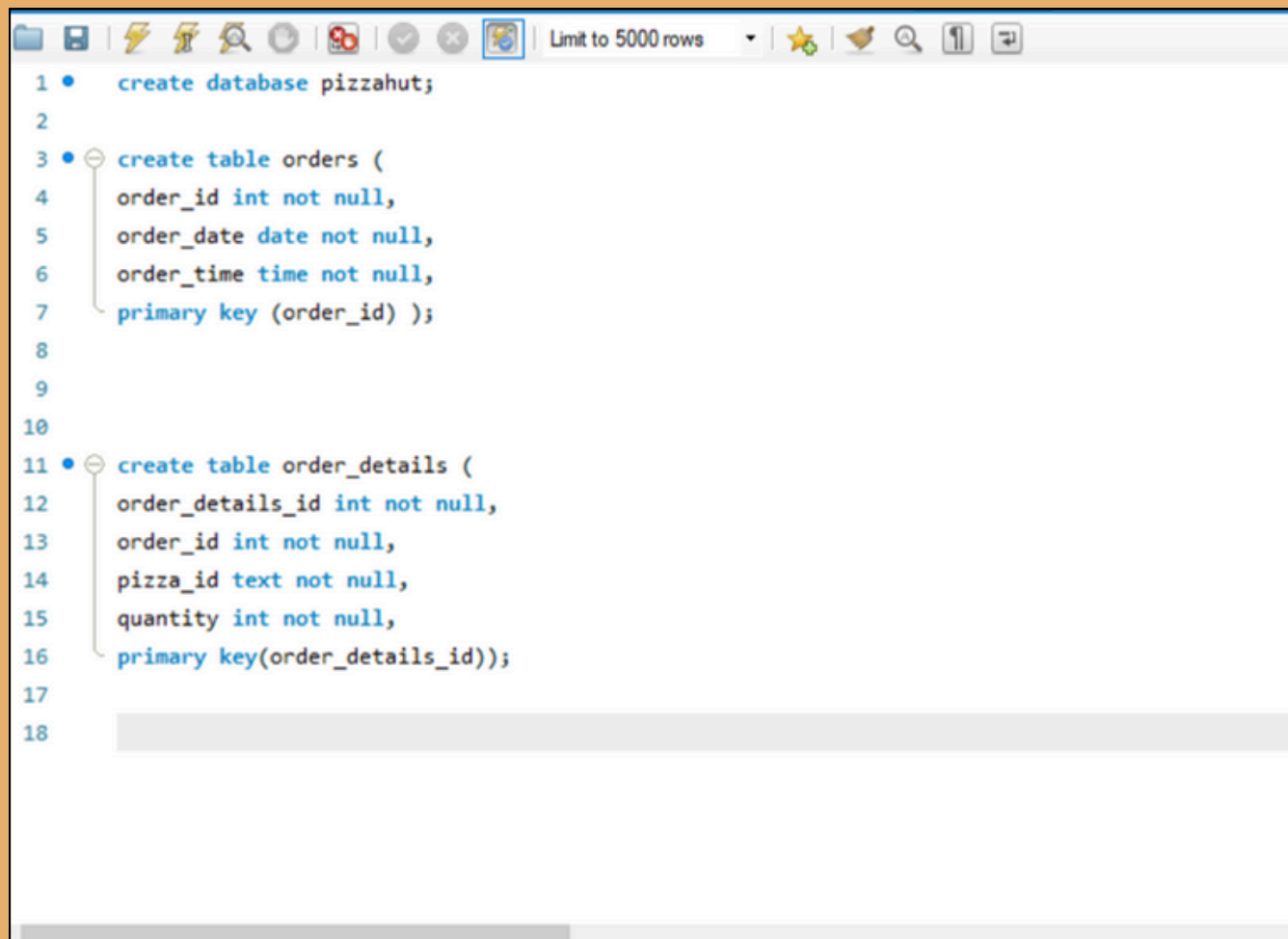
My first SQL Project

**EXPLORE A STRUCTURED
BREAKDOWN OF PIZZA SALES
DATA THROUGH A SERIES OF SQL
QUERIES—RANGING FROM
FOUNDATIONAL INSIGHTS TO
ADVANCED ANALYTICS. THIS
SECTION DEMONSTRATES
PRACTICAL QUERY SKILLS
APPLIED TO REAL-WORLD
BUSINESS QUESTIONS,
DELIVERING CLEAR, ACTIONABLE
INSIGHTS AT EVERY LEVEL.**

Pizza Order Data Model



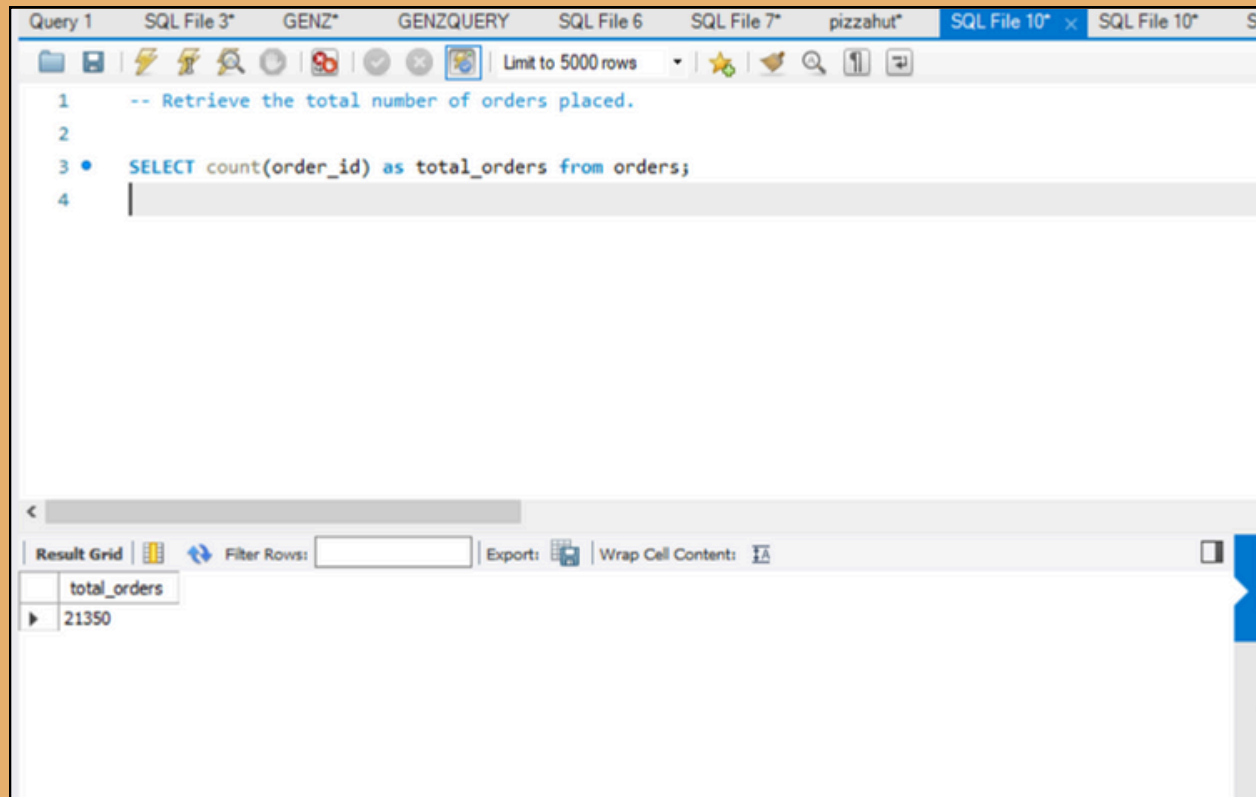
Creating a Database



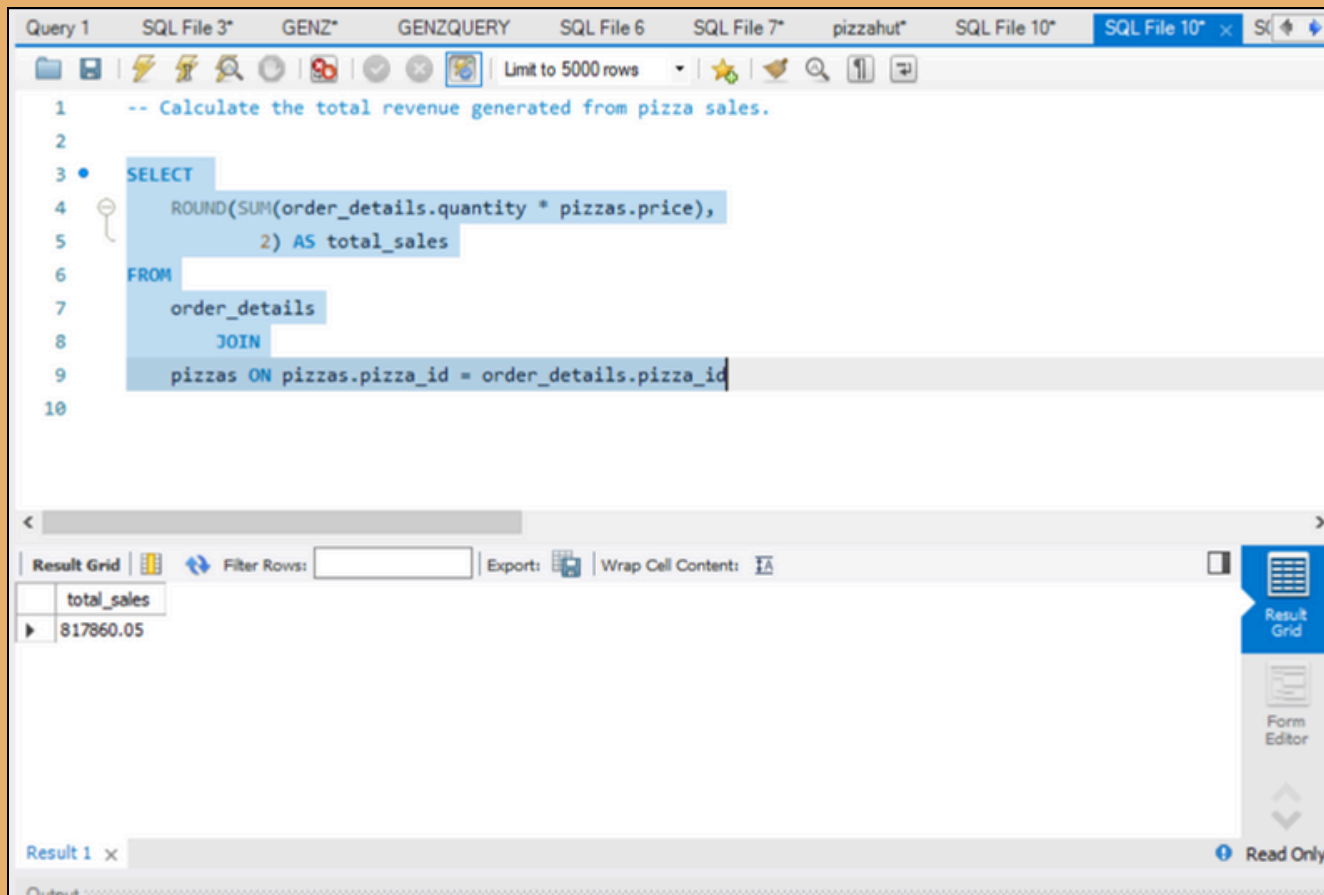
The image shows a screenshot of a SQL IDE window. The title bar includes icons for file operations and a dropdown menu set to 'Limit to 5000 rows'. The main text area contains SQL code for creating a database and two tables. Line numbers 1 through 18 are visible on the left margin. The code is as follows:

```
1 • create database pizzahut;  
2  
3 • create table orders (  
4     order_id int not null,  
5     order_date date not null,  
6     order_time time not null,  
7     primary key (order_id) );  
8  
9  
10  
11 • create table order_details (  
12     order_details_id int not null,  
13     order_id int not null,  
14     pizza_id text not null,  
15     quantity int not null,  
16     primary key(order_details_id));  
17  
18
```

1.Retrieve the total number of orders placed.



2. Calculate the total revenue generated from pizza sales.



The screenshot shows a SQL IDE window with multiple tabs. The active tab is 'SQL File 10*', which contains a SQL query. The query is as follows:

```
1  -- Calculate the total revenue generated from pizza sales.
2
3  SELECT
4      ROUND(SUM(order_details.quantity * pizzas.price),
5            2) AS total_sales
6  FROM
7      order_details
8      JOIN
9      pizzas ON pizzas.pizza_id = order_details.pizza_id
10
```

Below the query editor, the 'Result Grid' is visible, showing the result of the query:

total_sales
817860.05

The IDE interface includes a toolbar at the top with various icons for file operations, a 'Limit to 5000 rows' dropdown, and a 'Filter Rows' input field. The bottom right corner of the IDE shows a 'Read Only' status.



3. Identify the highest-priced pizza.

```
SQL File 3*  GENZ*  GENZQUERY  SQL File 6  SQL File 7*  pizzahut*  SQL File 10*  SQL File 10*  SQL File 11* x
Limit to 5000 rows
1  -- Identify the highest-priced pizza.
2
3  • SELECT
4      pizza_types.name, pizzas.price
5  FROM
6      pizza_types
7      JOIN
8      pizzas ON pizza_types.pizza_type_id = pizzas.pizza_type_id
9  ORDER BY pizzas.price DESC
10 LIMIT 1;
```

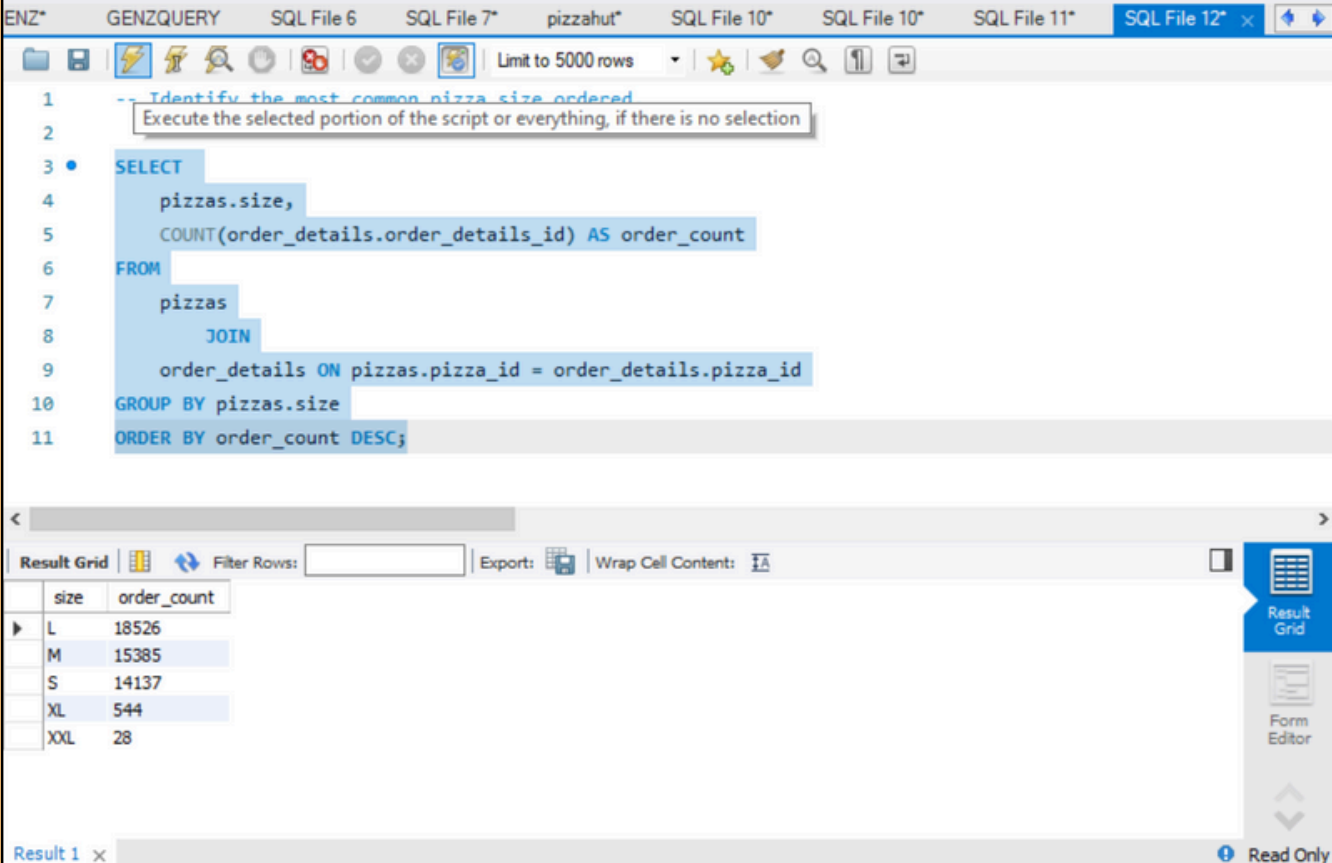
Result Grid

name	price
The Greek Pizza	35.95

Result 1 x Read Only



4. Identify the most common pizza size ordered.



The screenshot shows a SQL IDE window with multiple tabs. The active tab is 'SQL File 12'. The query editor contains the following SQL code:

```
1 -- Identify the most common pizza size ordered
2 Execute the selected portion of the script or everything, if there is no selection
3 • SELECT
4     pizzas.size,
5     COUNT(order_details.order_details_id) AS order_count
6 FROM
7     pizzas
8     JOIN
9     order_details ON pizzas.pizza_id = order_details.pizza_id
10 GROUP BY pizzas.size
11 ORDER BY order_count DESC;
```

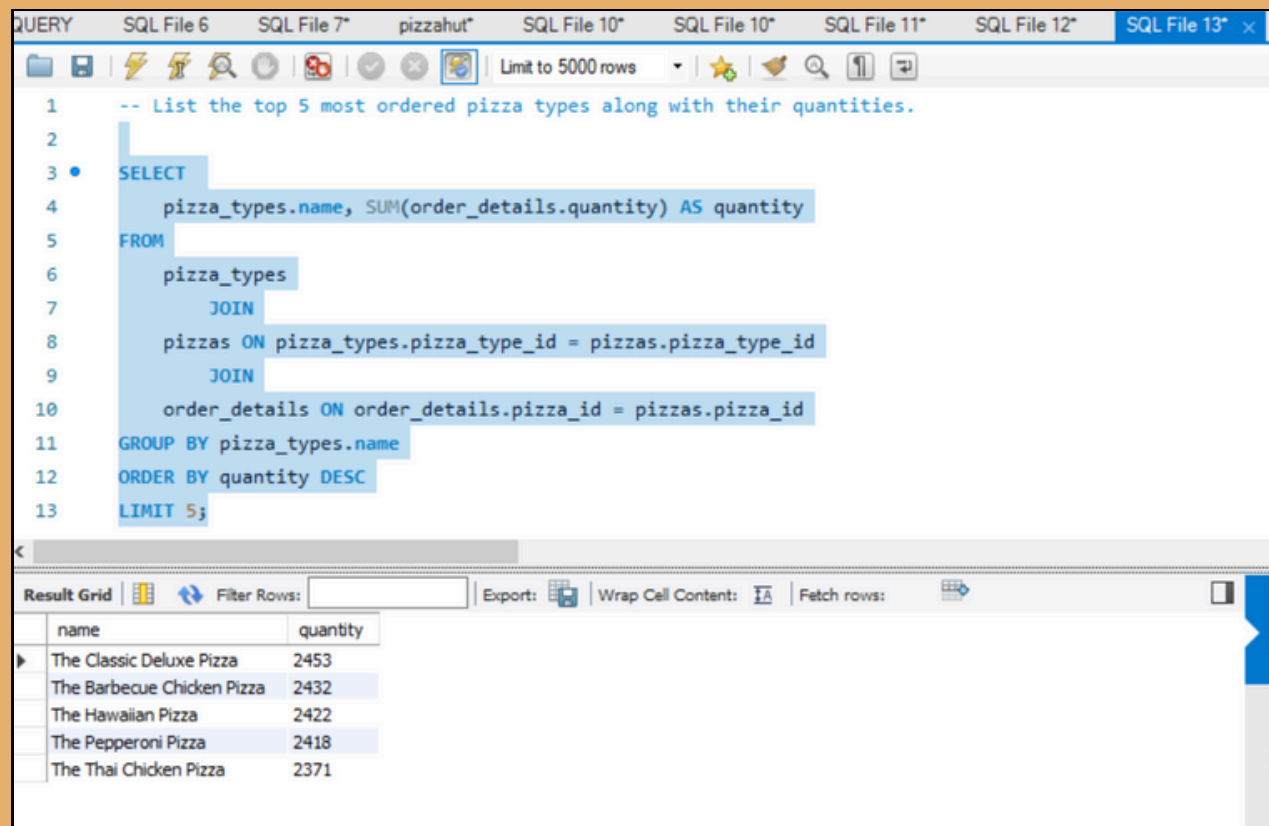
Below the query editor, the 'Result Grid' is displayed, showing the results of the query:

size	order_count
L	18526
M	15385
S	14137
XL	544
XXL	28

The 'Result Grid' tab is selected, and the results are shown in a table format. The 'L' size has the highest order count at 18,526.



5. List the top 5 most ordered pizza types along with their quantities.



The screenshot shows a SQL IDE window with a query editor and a results grid. The query is as follows:

```
-- List the top 5 most ordered pizza types along with their quantities.
SELECT
  pizza_types.name, SUM(order_details.quantity) AS quantity
FROM
  pizza_types
  JOIN
  pizzas ON pizza_types.pizza_type_id = pizzas.pizza_type_id
  JOIN
  order_details ON order_details.pizza_id = pizzas.pizza_id
GROUP BY pizza_types.name
ORDER BY quantity DESC
LIMIT 5;
```

The results grid displays the following data:

name	quantity
The Classic Deluxe Pizza	2453
The Barbecue Chicken Pizza	2432
The Hawaiian Pizza	2422
The Pepperoni Pizza	2418
The Thai Chicken Pizza	2371



6. Join the necessary tables to find the total quantity of each pizza category ordered.

```
File 6  SQL File 7*  pizzahut*  SQL File 10*  SQL File 10*  SQL File 11*  SQL File 12*  SQL File 13*  SQL File 14* x
Limit to 5000 rows
1  -- Join the necessary tables to find the total quantity of each pizza category ordered.
2
3  SELECT
4      pizza_types.category,
5      SUM(order_details.quantity) AS quantity
6  FROM
7      pizza_types
8      JOIN
9      pizzas ON pizza_types.pizza_type_id = pizzas.pizza_type_id
10     JOIN
11     order_details ON order_details.pizza_id = pizzas.pizza_id
12 GROUP BY pizza_types.category
13 ORDER BY quantity DESC;
```

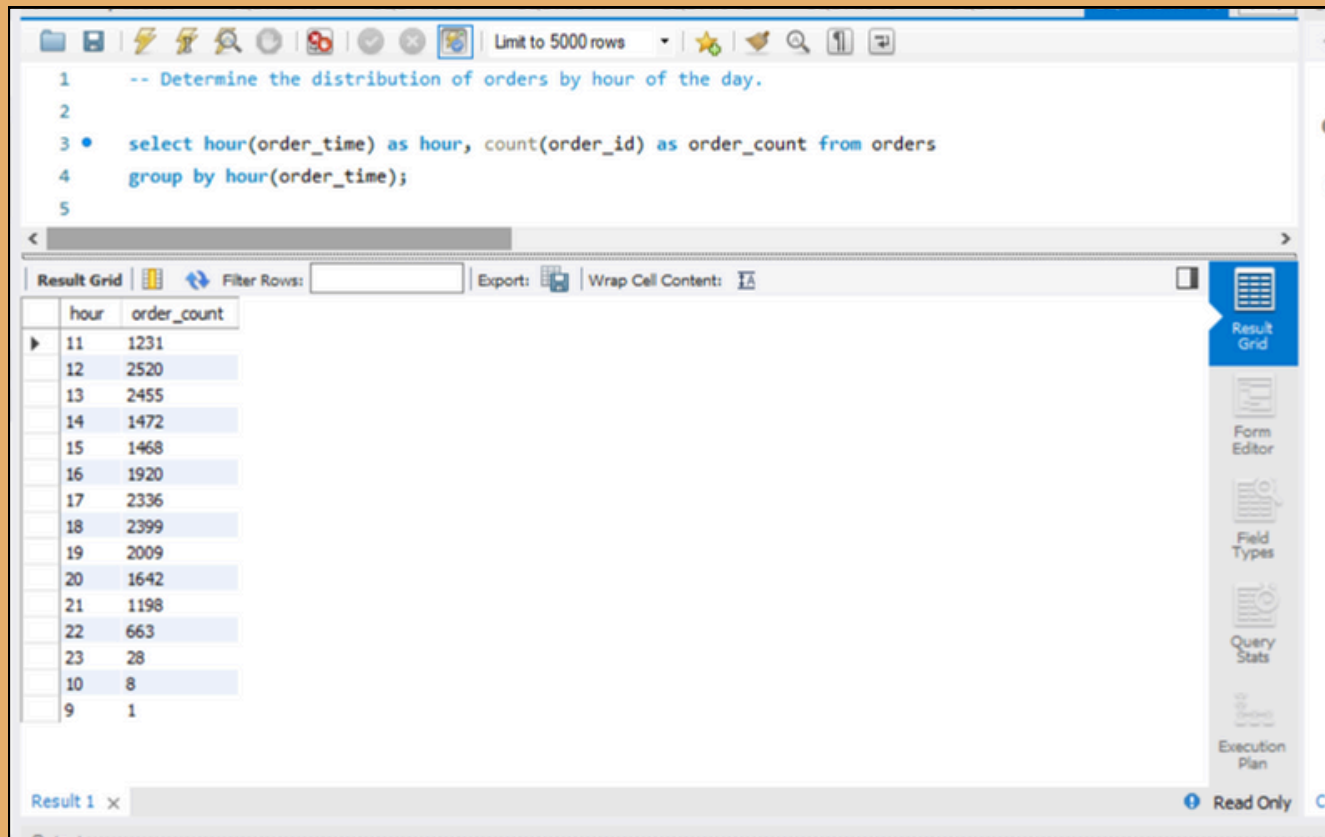
Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

category	quantity
Classic	14888
Supreme	11987
Veggie	11649
Chicken	11050

Result 1 x | Read Only



7. Determine the distribution of orders by hour of the day.



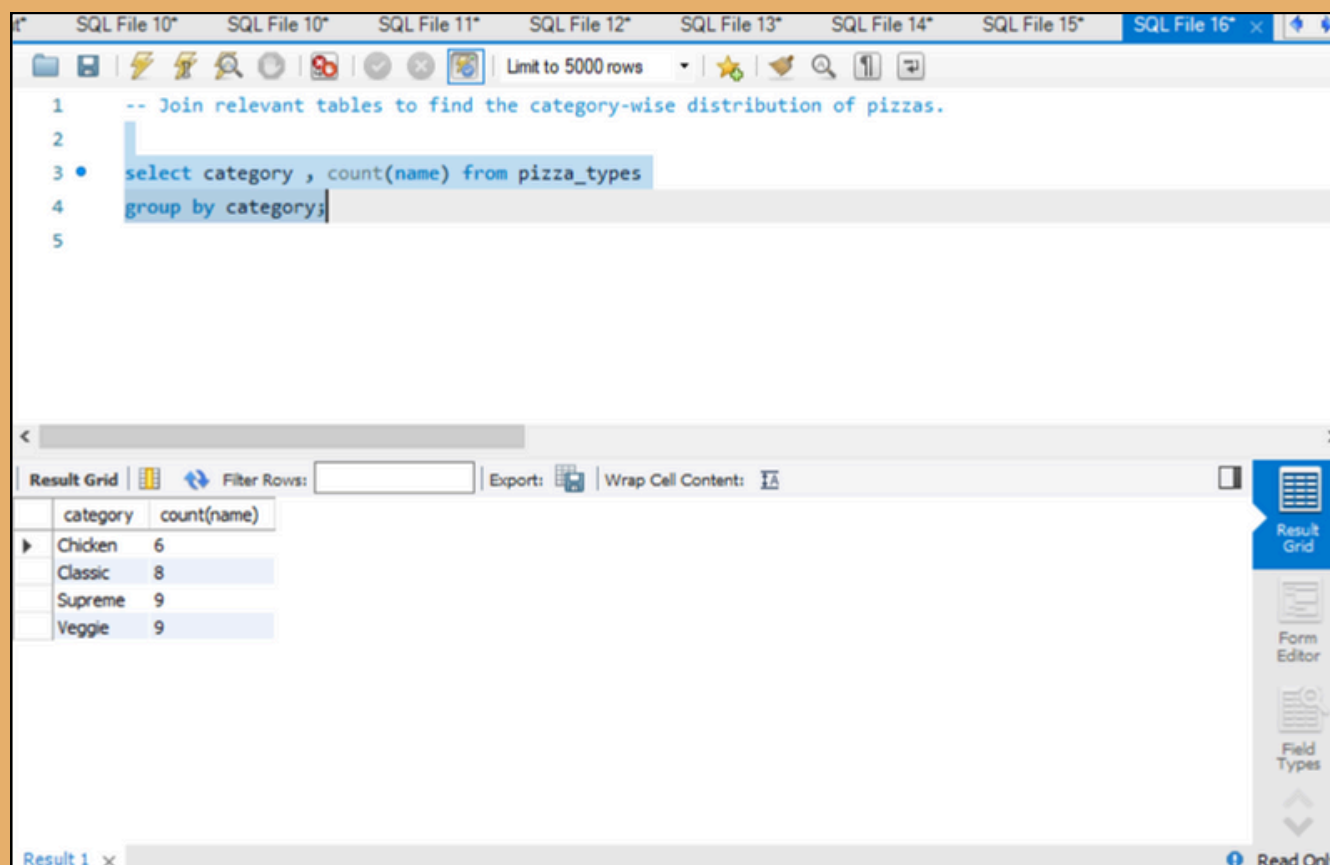
```
-- Determine the distribution of orders by hour of the day.

select hour(order_time) as hour, count(order_id) as order_count from orders
group by hour(order_time);
```

hour	order_count
11	1231
12	2520
13	2455
14	1472
15	1468
16	1920
17	2336
18	2399
19	2009
20	1642
21	1198
22	663
23	28
10	8
9	1



8. Join relevant tables to find the category-wise distribution of pizzas.



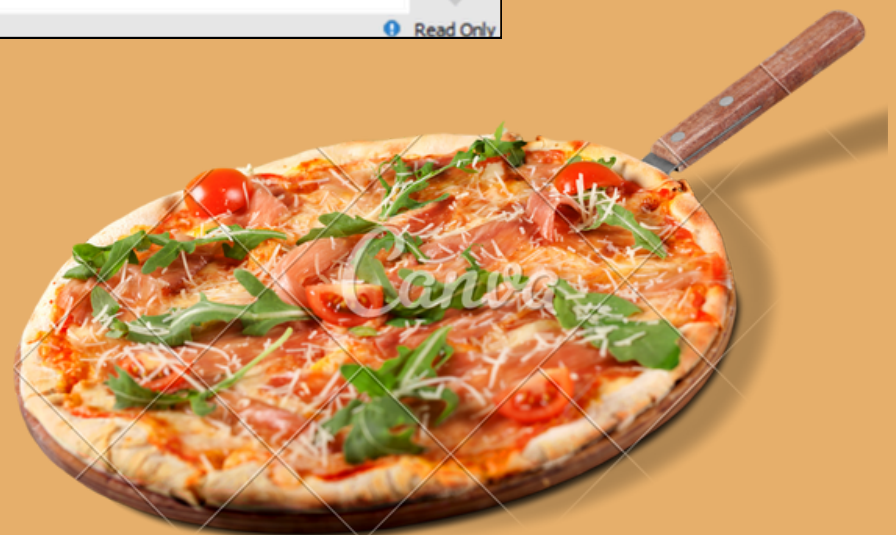
The screenshot shows a SQL IDE with multiple tabs labeled 'SQL File 10*' through 'SQL File 16*'. The active tab 'SQL File 16*' contains the following SQL query:

```
1  -- Join relevant tables to find the category-wise distribution of pizzas.
2
3  select category , count(name) from pizza_types
4  group by category;
```

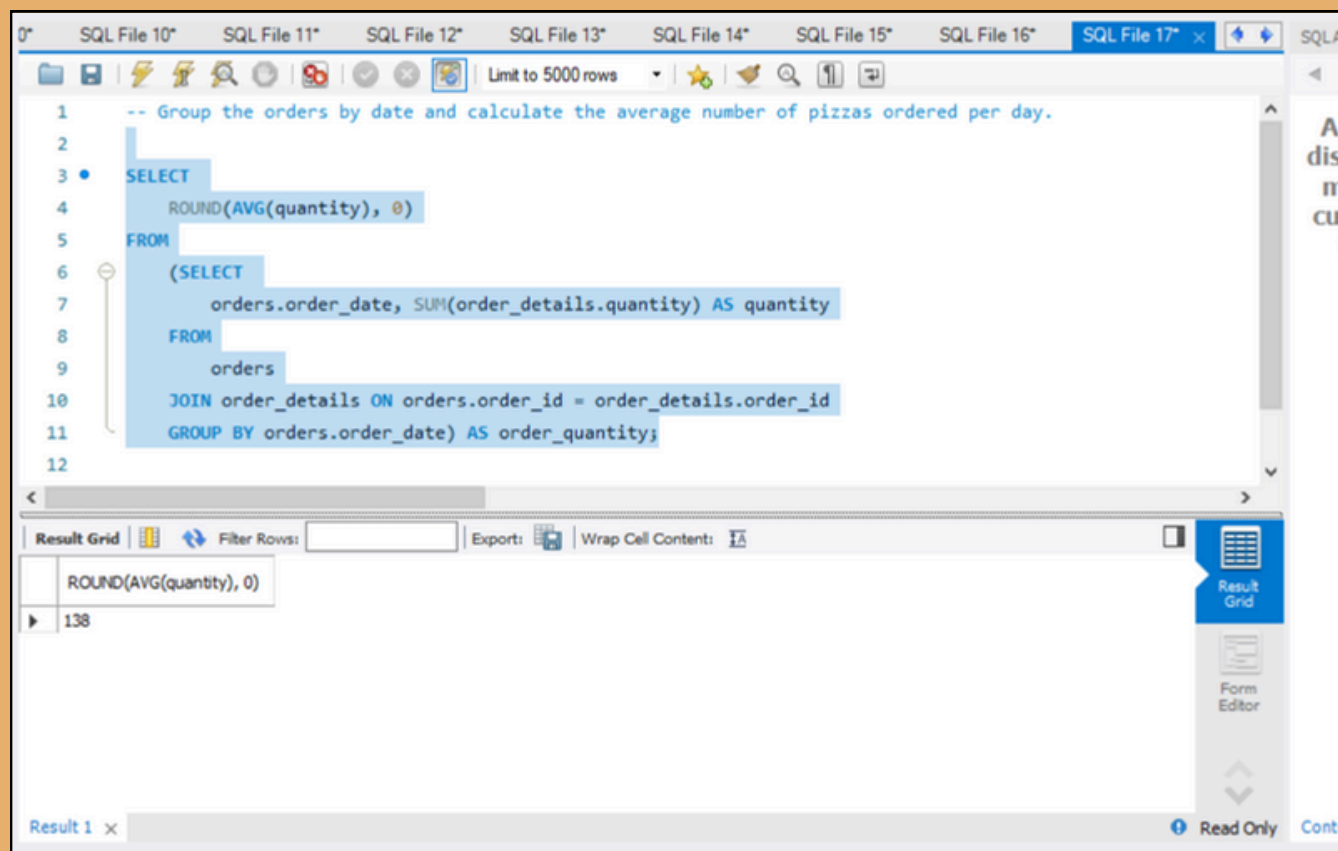
Below the query editor, the 'Result Grid' is displayed, showing the results of the query. The grid has two columns: 'category' and 'count(name)'. The results are as follows:

category	count(name)
Chicken	6
Classic	8
Supreme	9
Veggie	9

The IDE interface includes various toolbars and a sidebar on the right with options like 'Result Grid', 'Form Editor', and 'Field Types'.



9. Group the orders by date and calculate the average number of pizzas ordered per day.



The screenshot shows a SQL IDE with multiple tabs labeled 'SQL File 10*' through 'SQL File 17*'. The active tab is 'SQL File 17*'. The SQL editor contains the following query:

```
-- Group the orders by date and calculate the average number of pizzas ordered per day.
1
2
3 SELECT
4     ROUND(AVG(quantity), 0)
5 FROM
6     (SELECT
7         orders.order_date, SUM(order_details.quantity) AS quantity
8     FROM
9         orders
10        JOIN order_details ON orders.order_id = order_details.order_id
11        GROUP BY orders.order_date) AS order_quantity;
```

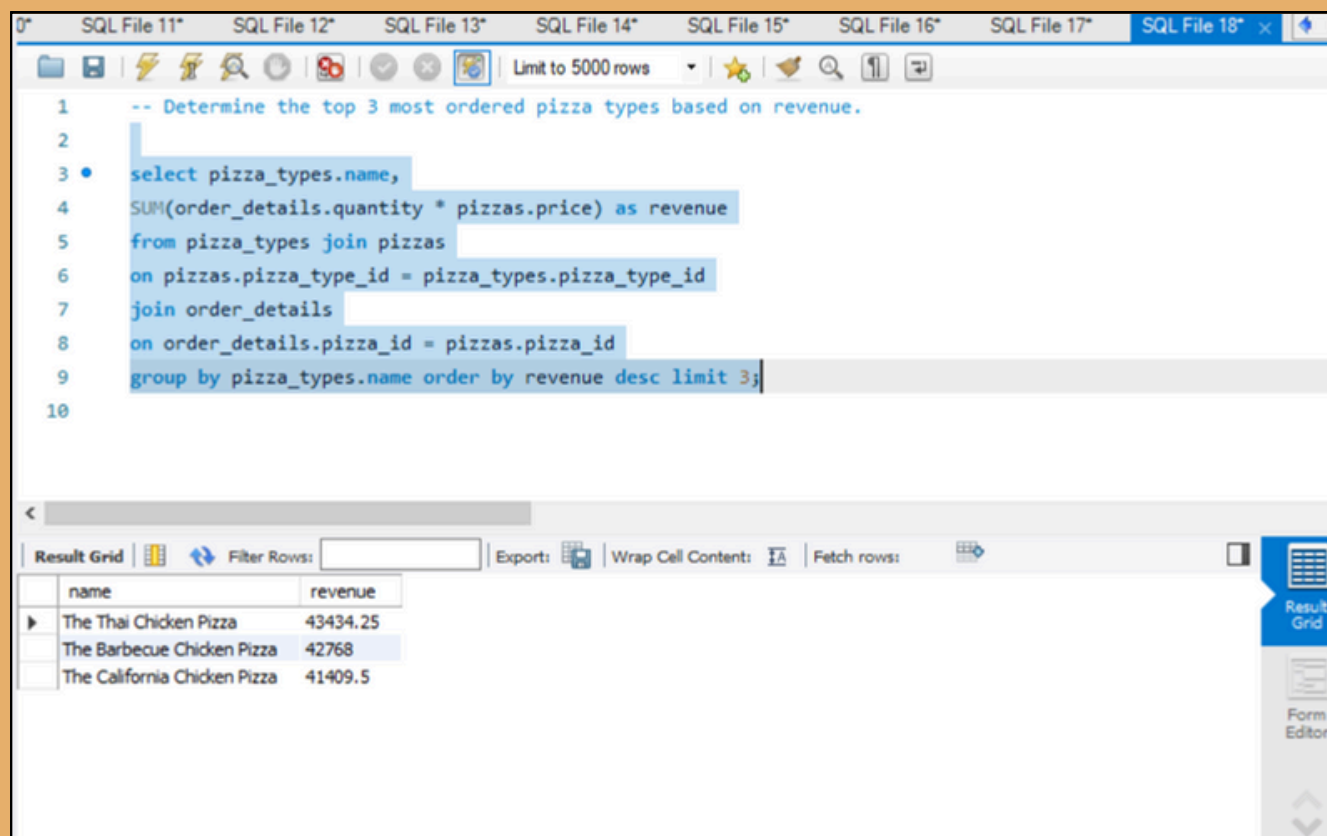
Below the editor, the 'Result Grid' is visible, showing the query result:

ROUND(AVG(quantity), 0)
138

The interface includes a toolbar with icons for file operations, a 'Limit to 5000 rows' dropdown, and a 'Filter Rows' input field. The bottom status bar indicates 'Result 1 x' and 'Read Only'.



10. Determine the top 3 most ordered pizza types based on revenue.



The screenshot shows a SQL IDE with multiple tabs. The active tab, 'SQL File 18*', contains the following SQL query:

```
1  -- Determine the top 3 most ordered pizza types based on revenue.
2
3  select pizza_types.name,
4  SUM(order_details.quantity * pizzas.price) as revenue
5  from pizza_types join pizzas
6  on pizzas.pizza_type_id = pizza_types.pizza_type_id
7  join order_details
8  on order_details.pizza_id = pizzas.pizza_id
9  group by pizza_types.name order by revenue desc limit 3;
```

Below the query editor, the 'Result Grid' is displayed, showing the top 3 pizza types by revenue:

name	revenue
The Thai Chicken Pizza	43434.25
The Barbecue Chicken Pizza	42768
The California Chicken Pizza	41409.5



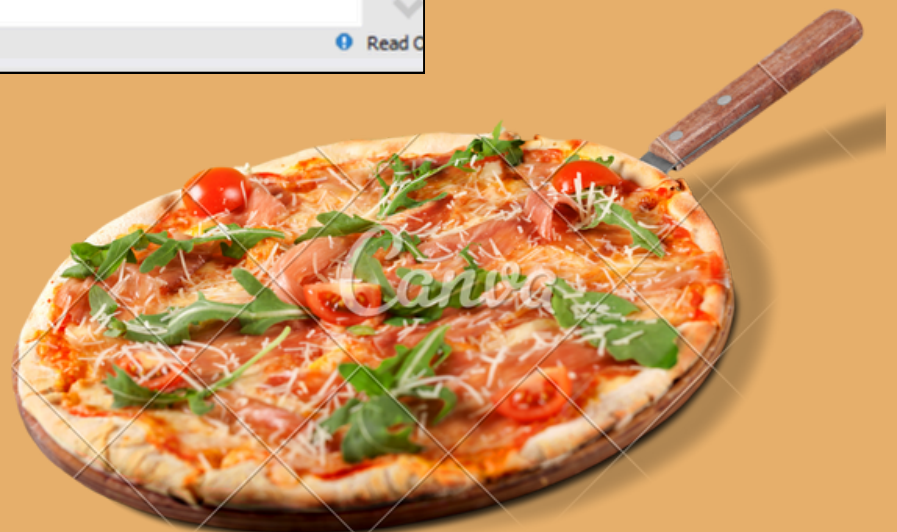
11. Calculate the percentage contribution of each pizza type to total revenue.

```
1  -- Calculate the percentage contribution of each pizza type to total revenue.
2
3  select pizza_types.category,
4  ROUND(sum( order_details.quantity*pizzas.price) / (SELECT
5  ROUND(SUM(order_details.quantity * pizzas.price),
6  2) AS total_sales
7  FROM
8  order_details
9  JOIN
10  pizzas ON pizzas.pizza_id = order_details.pizza_id)*100,2) as revenue
11  from pizza_types join pizzas
12  on pizza_types.pizza_type_id = pizzas.pizza_type_id
13  join order_details
14  on order_details.pizza_id = pizzas.pizza_id
15  group by pizza_types.category order by revenue desc;
```

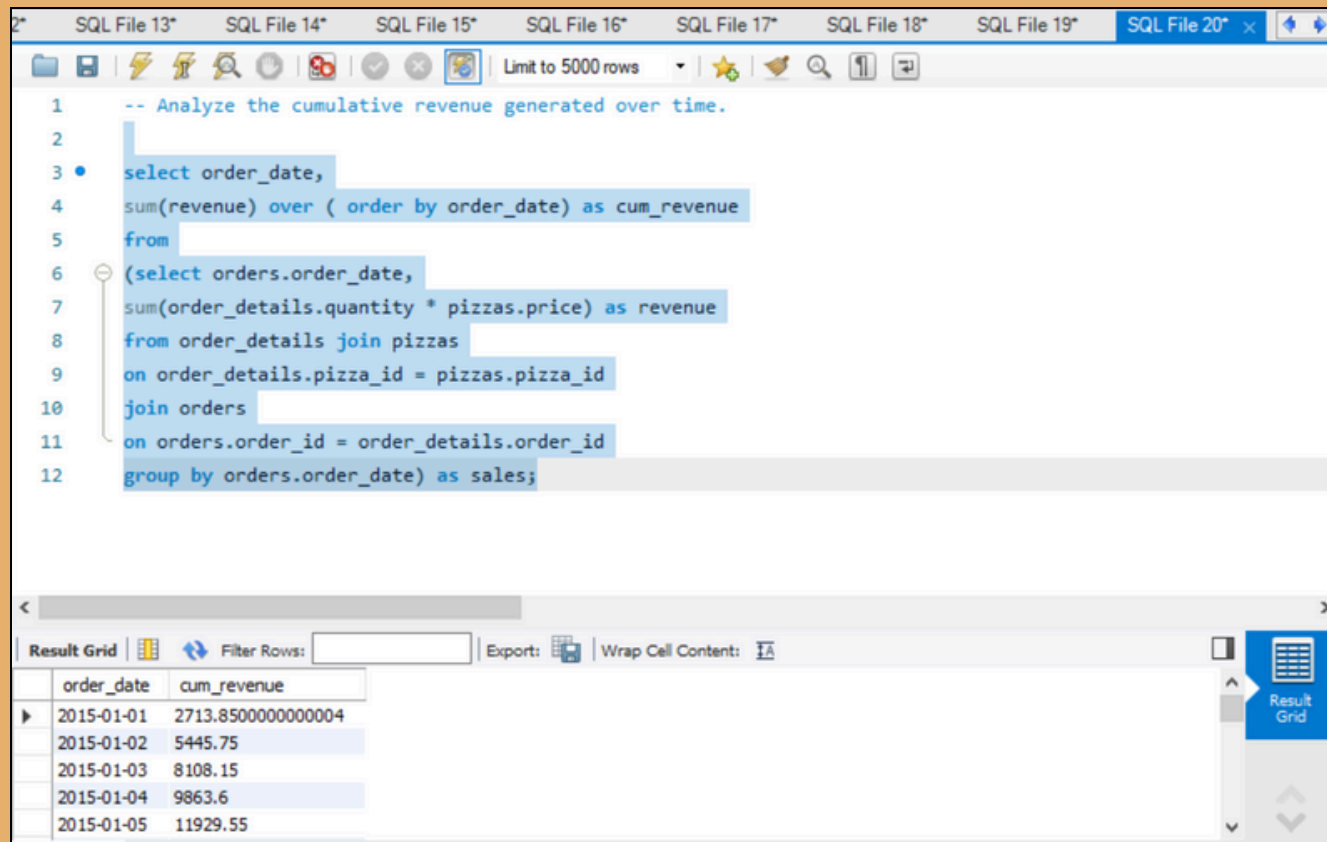
Result Grid

category	revenue
Classic	26.91
Supreme	25.46
Chicken	23.96
Veggie	23.68

Result 1 x



12. Analyze the cumulative revenue generated over time.



The screenshot shows a SQL IDE with a query editor and a result grid. The query is as follows:

```
-- Analyze the cumulative revenue generated over time.  
  
select order_date,  
sum(revenue) over ( order by order_date) as cum_revenue  
from  
(select orders.order_date,  
sum(order_details.quantity * pizzas.price) as revenue  
from order_details join pizzas  
on order_details.pizza_id = pizzas.pizza_id  
join orders  
on orders.order_id = order_details.order_id  
group by orders.order_date) as sales;
```

The result grid displays the following data:

order_date	cum_revenue
2015-01-01	2713.8500000000004
2015-01-02	5445.75
2015-01-03	8108.15
2015-01-04	9863.6
2015-01-05	11929.55



13. Determine the top 3 most ordered pizza types based on revenue for each pizza category.

```
SQL File 14*  SQL File 15*  SQL File 16*  SQL File 17*  SQL File 18*  SQL File 19*  SQL File 20*  SQL File 21* x
Limit to 5000 rows
1  -- Determine the top 3 most ordered pizza types based on revenue for each pizza category.
2
3  select name, revenue from
4  (select category, name, revenue,
5   rank() over(partition by category order by revenue desc) as rn
6   from
7   (select pizza_types.category, pizza_types.name,
8    sum((order_details.quantity) * pizzas.price) as revenue
9    from pizza_types join pizzas
10   on pizza_types.pizza_type_id = pizzas.pizza_type_id
11   join order_details
12   on order_details.pizza_id = pizzas.pizza_id
13   group by pizza_types.category , pizza_types.name) as a) as b
14  where rn<=3;
```

Result Grid

	name	revenue
▶	The Thai Chicken Pizza	43434.25
	The Barbecue Chicken Pizza	42768
	The California Chicken Pizza	41409.5
	The Classic Deluxe Pizza	The California Chicken Pizza
	The Hawaiian Pizza	32273.25



Thank You!

Appreciate you checking out my
first SQL project.

It's been a great learning
experience—and just the start.

More queries, more insights
ahead! Open to feedback and
suggestions as I keep learning.